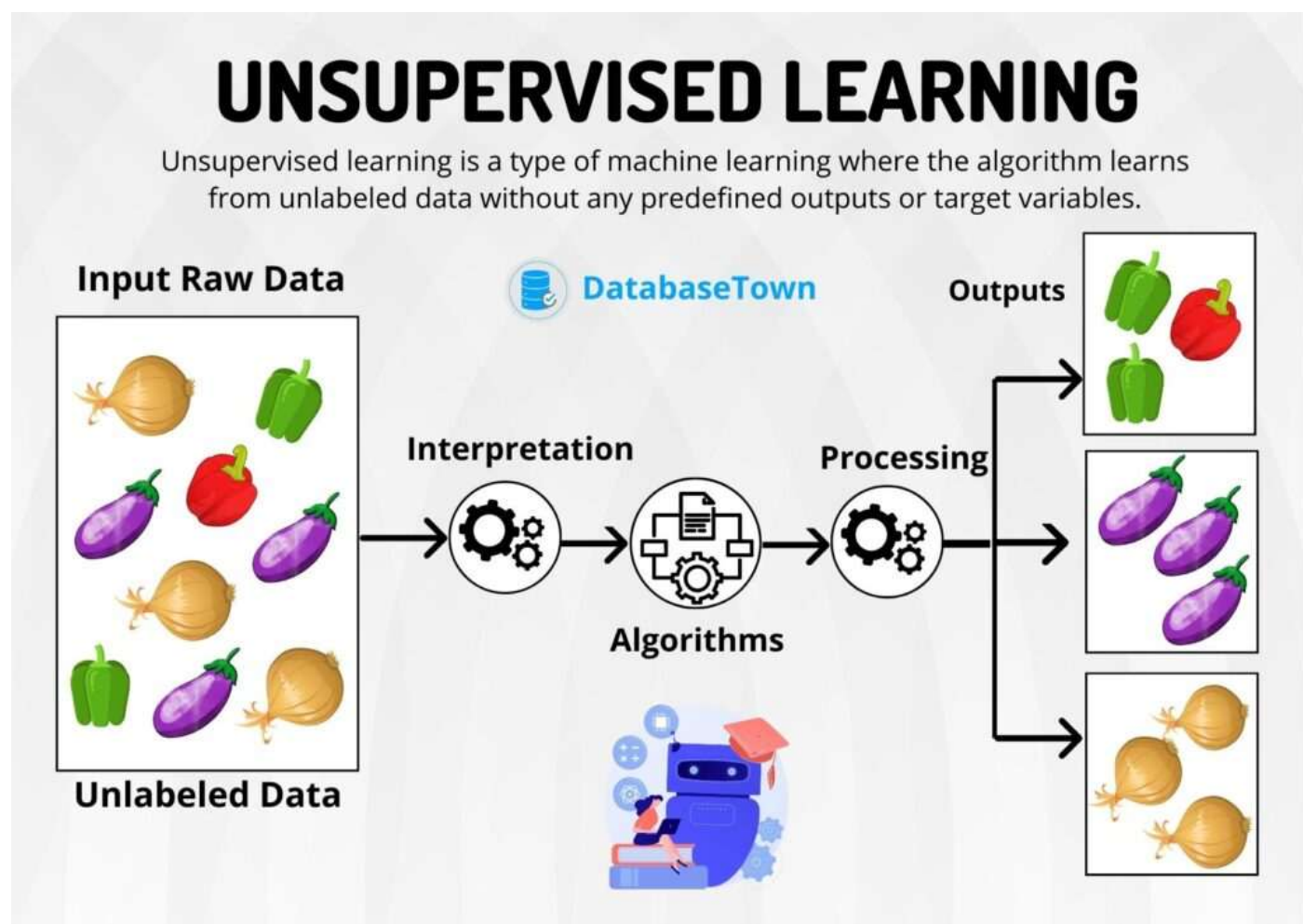
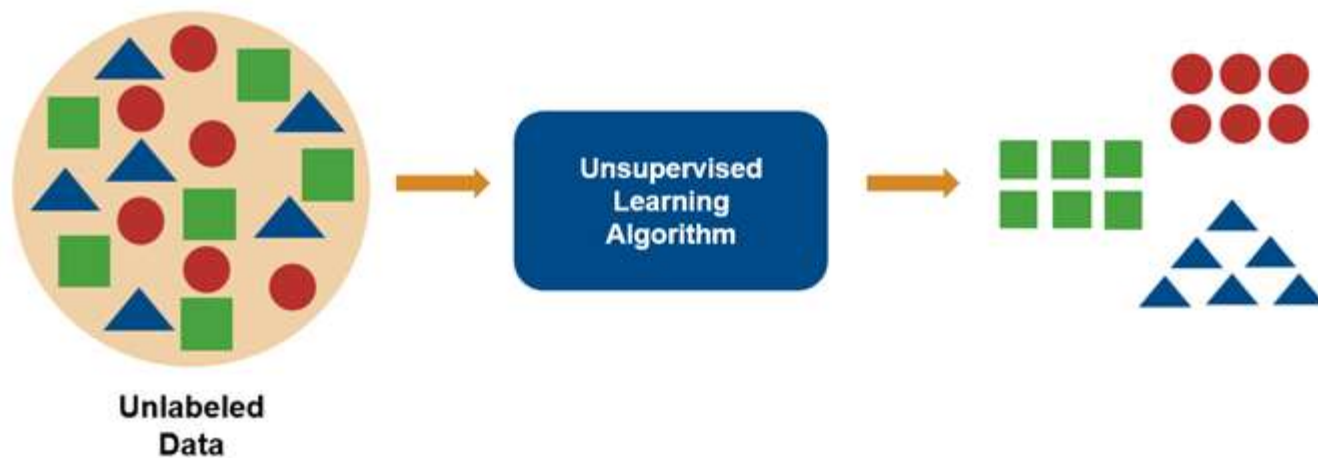


Introduction to Unsupervised Learning

Unsupervised Learning is a type of machine learning that involves training a model using **unlabelled data**. Unlike supervised learning, there is **no predefined output variable** or "correct answer."



The main goal is for the algorithm to autonomously discover and infer **hidden patterns, structures, and relationships** within the data, such as:

- Grouping similar data points together.
- Identifying underlying data distributions.
- Reducing the number of variables (features).

Key Characteristics:

- **Data Type:** Uses **unlabelled** or **untagged** input data.
- **Goal:** Exploration, pattern discovery, and data structure revelation.
- **Applications:** Customer segmentation, anomaly/fraud detection, data visualization, and market basket analysis.
- **Advantage:** Can work with raw data and discover insights that human labelers might miss.
- **Challenge:** Evaluating the model's performance can be more subjective as there are no ground-truth labels.

Types of Unsupervised Learning

Unsupervised learning is generally categorized into three main tasks:

1. Clustering

Clustering is the task of grouping a set of data points into clusters so that data points in the same cluster are **more similar** to each other than to those in other clusters. It helps find natural groupings in raw, unclassified data.

Type of Clustering	Description	Key Algorithm Examples
Exclusive/Partitioning	Data points belong exclusively to only one cluster (hard clustering).	K-Means , K-Medoids
Hierarchical	Builds a hierarchy of clusters, represented as a tree-like diagram (dendrogram). It can be: - Agglomerative (Bottom-up: starts with individual points and merges clusters) - Divisive (Top-down: starts with one cluster and splits it)	Agglomerative Clustering

Density-Based	Defines clusters as areas of higher density separated by areas of lower density (can find clusters of arbitrary shape).	DBSCAN (Density-Based Spatial Clustering of Applications with Noise)
Probabilistic/Model-Based	Assigns data points to clusters based on the probability of belonging to each cluster.	Gaussian Mixture Models (GMM)

2. Association Rule Learning (Association)

Association Rule Learning is a rule-based method for discovering interesting relationships (dependencies) between variables in large datasets. It identifies patterns in the form of **"if-then" rules**.

- **Goal:** Discover strong relationships between items.
- **Example:** In a grocery store, discovering that people who buy milk often also buy cereal. This is known as **Market Basket Analysis**.
- **Key Algorithms:** Apriori, FP-Growth, Eclat.

3. Dimensionality Reduction

Dimensionality Reduction is the process of reducing the number of random variables (features) in a dataset while retaining most of the important information. This is crucial for dealing with high-dimensional data, which can be computationally intensive and suffer from the **"Curse of Dimensionality"**.

- **Goal:** Simplify data, reduce noise, improve visualization, and speed up computation.

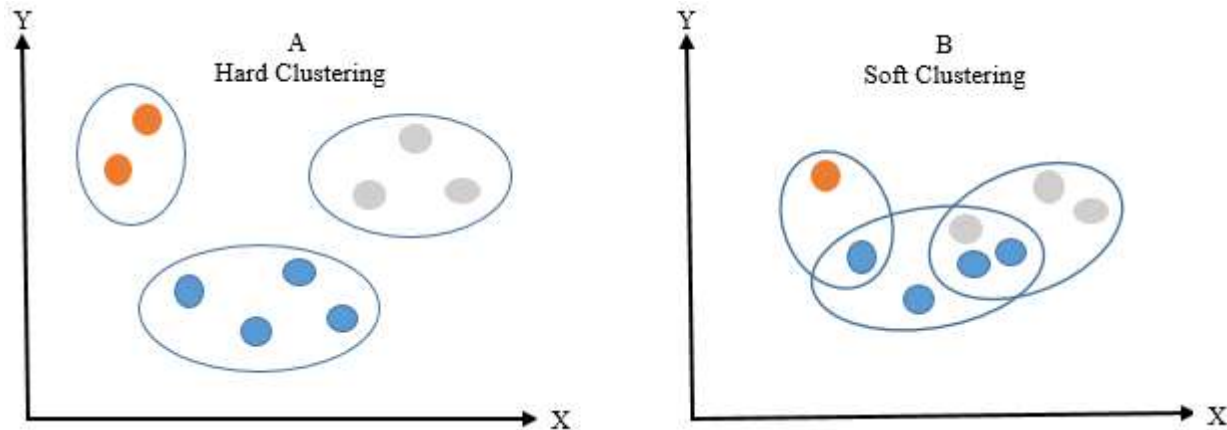
Two Main Approaches:

- **Feature Selection:** Selects a subset of the original features.
- **Feature Extraction:** Transforms the data into a new, smaller set of features (components).
- **Key Algorithms:**
 - **Principal Component Analysis (PCA):** A linear technique that identifies new dimensions (principal components) that capture the maximum variance in the data.

- **t-distributed Stochastic Neighbor Embedding (t-SNE):** A non-linear technique excellent for visualizing high-dimensional data in 2 or 3 dimensions.

Hard Clustering Vs Soft Clustering

The key difference between Hard and Soft Clustering lies in the **way they assign data points to clusters**.



- **Hard Clustering** says: "You belong 100% to **this one group**, and 0% to all others." (Like a strict club membership.)
- **Soft Clustering** says: "You belong a little bit to **all groups**, but mostly to this one, and here's the score for each." (Like having multiple part-time jobs.)

1. Hard Clustering:

Hard clustering, also called **exclusive clustering**, assigns each data point to **exactly one cluster**. The boundaries between clusters are **crisp** (clear-cut).

Feature	Description
Membership Rule	Strict/Binary (1 or 0): A data point either fully belongs to a cluster (1) or it doesn't (0).
Cluster Overlap	None: Clusters are non-overlapping.
Output	A single Cluster ID for every data point.
Common Algorithm	K-Means: Each point is assigned to the nearest cluster center (centroid).

When to Use	When you need clear, definitive segmentation, and the data points are distinct with little ambiguity (e.g., classifying an email as Spam or Not Spam).
--------------------	--

Hard Clustering Example: Customer Segmentation

Imagine a car dealership wants to group its customers to target them with different sales strategies.

- **Cluster 1: "Budget Buyer"** (Focus on value and fuel efficiency)
- **Cluster 2: "Luxury Seeker"** (Focus on features and comfort)

In hard clustering (like K-Means), Customer A is placed exclusively in **Cluster 2**, and Customer B is placed exclusively in **Cluster 1**. There is no partial membership.

2. Soft Clustering:

Soft clustering, also called **fuzzy clustering** or **probabilistic clustering**, assigns a **degree of membership** or **probability** to each data point for *every* cluster. The boundaries between clusters are **fuzzy** (allow for overlap).

Feature	Description
Membership Rule	Partial/Probabilistic (0 to 1): A data point has a value between 0 and 1 for each cluster, showing how strongly it belongs. The scores for all clusters usually sum to 1.0.
Cluster Overlap	Allowed: Data points on the boundary naturally belong to multiple groups.
Output	A vector of membership scores for every data point across all clusters.
Common Algorithm	Fuzzy C-Means (FCM), Gaussian Mixture Models (GMM).
When to Use	When clusters are known to overlap, or when you need to capture ambiguity or mixed characteristics in the data (e.g., grouping news articles that cover multiple topics).

Soft Clustering Example: Customer Segmentation

Using the same car dealership example, a soft clustering algorithm (like Fuzzy C-Means) provides a more nuanced view:

Customer	Budget Buyer Score	Luxury Seeker Score	Interpretation
Customer A	0.10	0.90	Primarily a Luxury Seeker.
Customer C	0.45	0.55	Mixed Buyer. They are almost equally likely to be a Budget Buyer or a Luxury Seeker, perhaps looking for a high-end small car.

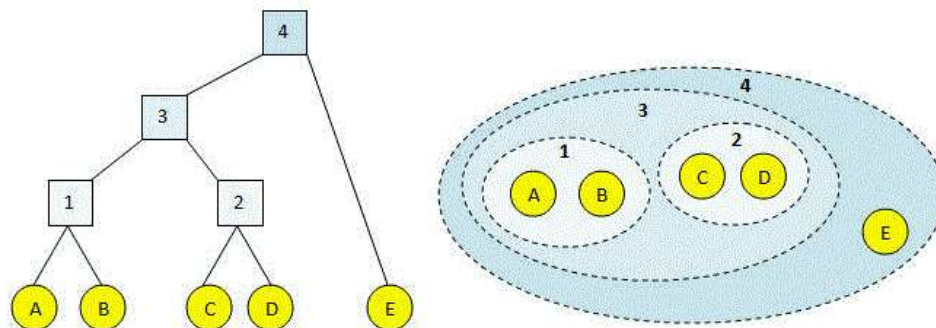
Customer C is the point of interest here—they don't fit perfectly into a single box, and soft clustering handles this **ambiguity** by quantifying their affinity to both groups.

Hierarchical Clustering:

Hierarchical Clustering is an unsupervised machine learning method that builds a tree-like structure, called a **dendrogram**, to show how data points are grouped into nested clusters based on their similarity. You don't need to specify the number of clusters beforehand.

There are two main types:

1. Agglomerative Clustering (Bottom-Up)

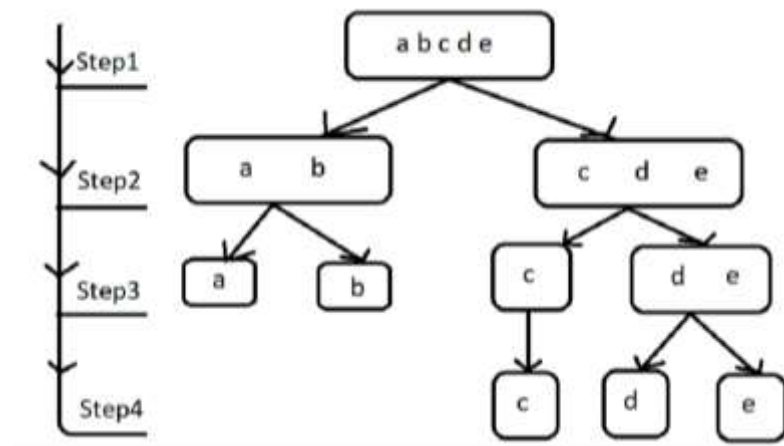


This is the **"start small, merge up"** approach. It begins with every data point as its own cluster and then repeatedly merges the closest clusters until all points are in one big cluster.

Steps:

1. **Start:** Treat **each data point** as its own single-point cluster.
 - *Example: If you have 5 data points, you start with 5 clusters.*
2. **Measure Distance:** Calculate the "**distance**" (or dissimilarity) between every pair of current clusters. This distance is calculated using a **linkage method** (e.g., measuring the distance between the closest two points in the clusters, the farthest two points, or the average distance).
3. **Merge:** Find the **two closest (most similar) clusters** and merge them into a single, new cluster.
4. **Repeat:** Go back to Step 2 and repeat the process (re-calculating all necessary distances and merging the closest pair) until **only one cluster remains** (the root of the tree).
5. **Result:** The entire history of merges is visualized in a **dendrogram**, which you can "cut" at any level to decide your final number of clusters.

2. Divisive Clustering (Top-Down)



This is the "**start big, split down**" approach. It begins with all data points in a single cluster and then repeatedly splits the clusters into smaller, more distinct sub-clusters.

Steps:

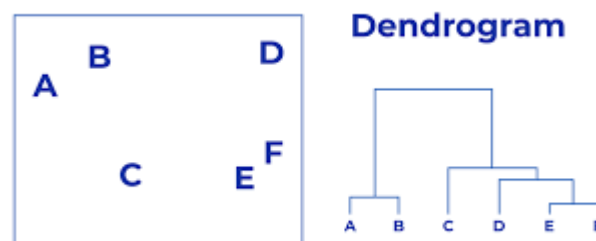
1. **Start:** Treat **all data points** as one single cluster.
2. **Split:** Choose the "**messiest**" or **most heterogeneous** cluster (the one with the most differences between its points) and divide it into two smaller, more homogeneous sub-clusters.

3. **Repeat:** Go back to Step 2 and repeat the process on the resulting clusters (choosing the "messiest" one to split) until **each data point is in its own cluster** (or you meet some other stopping condition, like a desired number of clusters).
4. **Result:** The history of splits is also shown in a **dendrogram**, read from the top down.

Feature	Agglomerative Clustering	Divisive Clustering
Approach	Bottom-Up (starts with individuals)	Top-Down (starts with the whole group)
Process	Merging the closest clusters	Splitting the most different/heterogeneous cluster
Starting Point	Each data point is a cluster	All data points form one cluster
Ending Point	All points in one cluster	Each data point is its own cluster
Commonality	More common and easier to implement	Less common (computationally more complex)

Dendrogram Analysis:

Dendrogram analysis is a technique used in **data science and statistics** to visualize the results of **hierarchical clustering**. A dendrogram is a **tree-like diagram** that illustrates the steps taken to group individual data points into progressively larger clusters. It allows an analyst to see the relationships and the degree of similarity/dissimilarity between all items in a dataset.



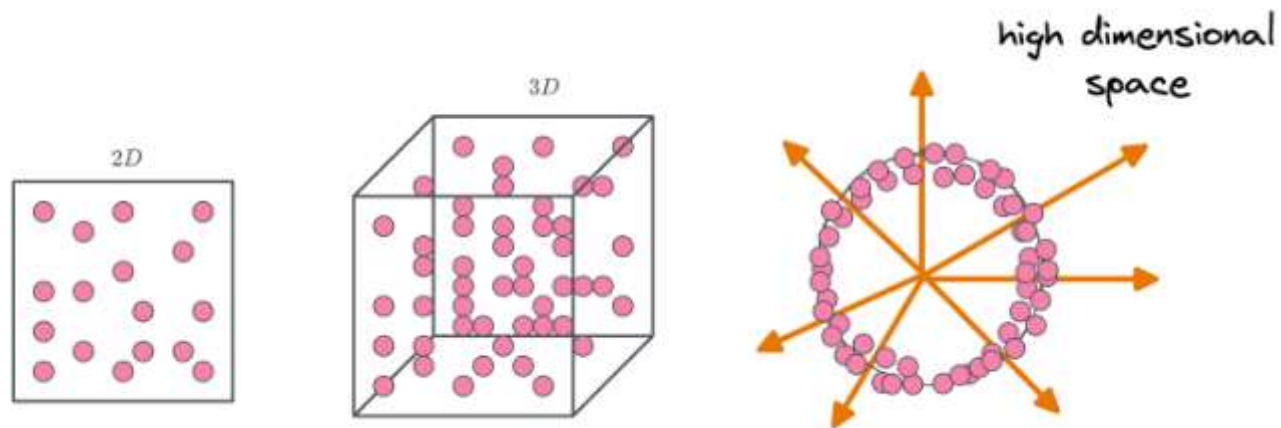
Key Concepts and Components

A dendrogram is built upon the following technical concepts:

- **Hierarchical Clustering:** This is the process that creates the dendrogram. It can be:

- **Agglomerative (Bottom-Up):** Starts with each data point as its own cluster and then successively merges the closest clusters until all points are in one large cluster. This is the most common approach.
- **Divisive (Top-Down):** Starts with one cluster containing all data points and recursively splits the most dissimilar clusters until each point is separate.
- **Leaves (Terminal Nodes):** These are the individual data points or objects being clustered. They form the base of the tree (typically on the horizontal axis).
- **Branches (Clades):** These are the lines that connect the leaves and clusters, showing the mergers that occurred during the clustering process.
- **Height/Distance Axis (Vertical Axis):** The height of the horizontal line where two clusters merge indicates the **distance** or **dissimilarity** between those clusters. The lower the merger, the more similar the items or clusters are.
 - In an agglomerative (bottom-up) dendrogram, this height increases as you move up the tree, reflecting greater dissimilarity at higher-level merges.
- **Linkage Method:** This is the rule used to calculate the distance between two existing clusters to decide which ones to merge next. Common methods include:
 - **Single Linkage (Minimum):** Uses the distance between the two *closest* members of the clusters being compared.
 - **Complete Linkage (Maximum):** Uses the distance between the two *farthest* members of the clusters being compared.
 - **Average Linkage:** Uses the average distance between all pairs of members from the two clusters.
 - **Ward's Method:** Merges the pair of clusters that results in the minimum increase in the total within-cluster variance.

Curse of Dimensionality:



The **Curse of Dimensionality** is a term describing the various problems that arise when working with datasets in **high-dimensional spaces** (data with a very large number of features or variables). As the number of dimensions increases, the complexity of the data space grows exponentially, causing machine learning models to struggle with performance, efficiency, and generalization.

Core Problems for Machine Learning Models

The high number of dimensions fundamentally alters the properties of the data space, leading to several critical issues:

- **Data Sparsity:**
 - As dimensions increase, the volume of the space grows exponentially. To maintain the same data density, the number of required data points must also grow exponentially.
 - With a fixed dataset size, the data points become increasingly **sparse** (spread out), making most of the high-dimensional space empty. This makes it difficult for algorithms to find enough local neighbours to establish reliable patterns.
- **Distance Concentration:**
 - In low-dimensional space, the distance to the nearest neighbour is clearly distinct from the distance to the farthest neighbour.
 - In high-dimensional space, the relative difference between these distances tends to **converge**. All data points become almost equally distant from one another.

- **Impact:** Distance-based algorithms like **k-Nearest Neighbours (k-NN)** and **k-Means Clustering** lose their effectiveness because proximity is no longer a meaningful measure of similarity.
- **Overfitting:**
 - When the number of features (p) is large relative to the number of samples (N), the model can easily fit to the **noise** and random fluctuations in the training data rather than the true underlying relationship.
 - The model achieves high accuracy on the training set but performs poorly on new, unseen data, which is the definition of **poor generalization**.
- **Increased Computational Cost:**
 - The exponential growth in the required data and the complexity of calculations (e.g., matrix operations) mean that training time and memory usage increase dramatically, making the process resource-intensive and slow.

Solutions: Dimensionality Reduction

The primary strategy to mitigate the curse of dimensionality is **Dimensionality Reduction**, which involves reducing the number of features while retaining the most important information.

Technique	Description	Purpose
Feature Selection	Identifies and removes irrelevant or redundant features from the original set.	Keeps only the most relevant original features (e.g., using correlation, filter methods, or LASSO regularization).
Feature Extraction	Creates a new, smaller set of features (components) that are linear or non-linear combinations of the original features.	Transforms the data into a lower-dimensional space.
Regularization	Adds a penalty term to the loss function based on the magnitude of the model's coefficients (e.g., L1/L2 regularization).	Penalizes overly complex models, discouraging them from fitting to noise, which is crucial in high dimensions.

Dimensionality Reduction Techniques: Principal Component Analysis (PCA), Feature Engineering, Feature Extraction & Selection:

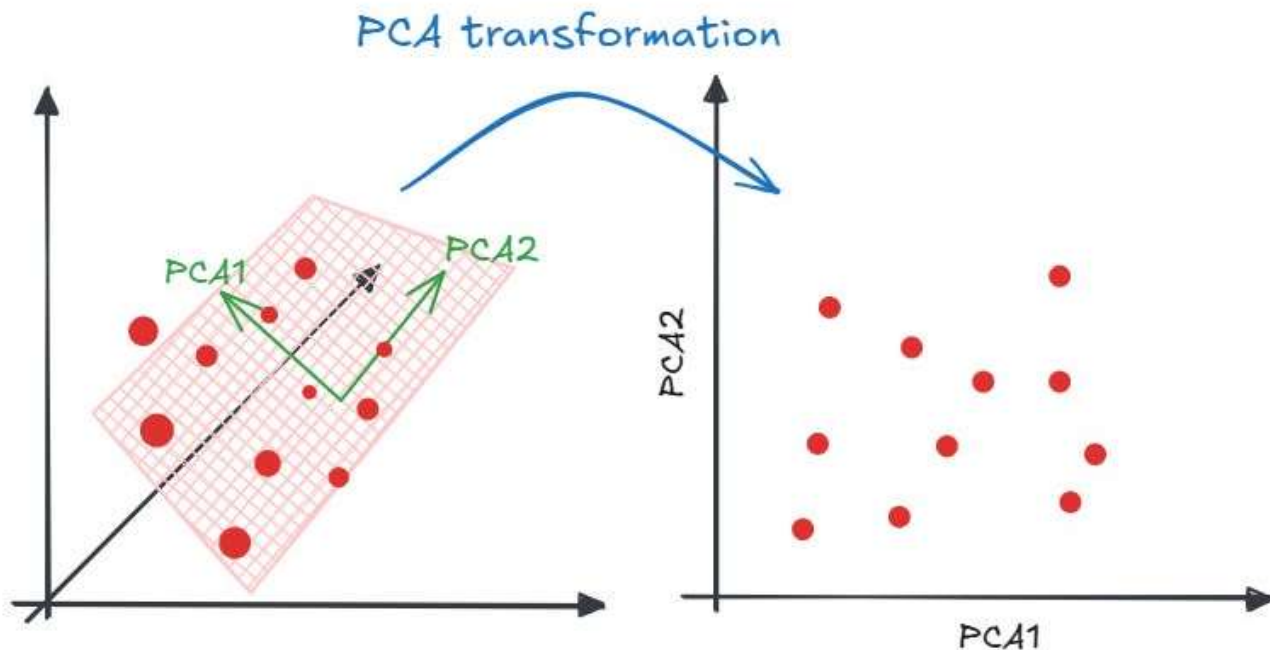
Dimensionality Reduction techniques are crucial in machine learning to combat the Curse of Dimensionality by reducing the number of input variables, which improves model performance, speed, and interpretability.

These techniques generally fall into three main categories: Feature Selection, Feature Extraction and the overarching process of Feature Engineering.

Principal Component Analysis:

Principal Component Analysis (PCA) is the most common **linear Feature Extraction** technique. It is an **unsupervised** algorithm that transforms the original set of possibly correlated features into a new, smaller set of **uncorrelated** variables called **Principal Components (PCs)**, while retaining most of the variance (information) present in the original dataset.

The process is "unsupervised" because it does not require an output or target variable (labels) to learn the data's underlying structure. It only looks at the input features.



Why PCA unsupervised:

- **Unsupervised:** PCA uses only the feature matrix X (no labels) to find directions of maximum variance.

- **What it does:** finds orthogonal axes (principal components) that capture most of the variance. Each PC is a linear combination of original features (the **loadings**).
- **How it's computed:** center the data → compute covariance (or use SVD) → eigenvectors/eigenvalues (eigenvectors = directions, eigenvalues → explained variance).
- **Interpretation:** `explained_variance_ratio_` tells you how much of total variance each PC explains. Use the cumulative explained variance to pick number of components (e.g., keep components that explain $\geq 95\%$ of variance).
- **Important:** **standardize** features before PCA (PCA is scale-sensitive). Outliers and non-linear structure can mislead PCA – use Kernel PCA, t-SNE, or UMAP for non-linear structure.

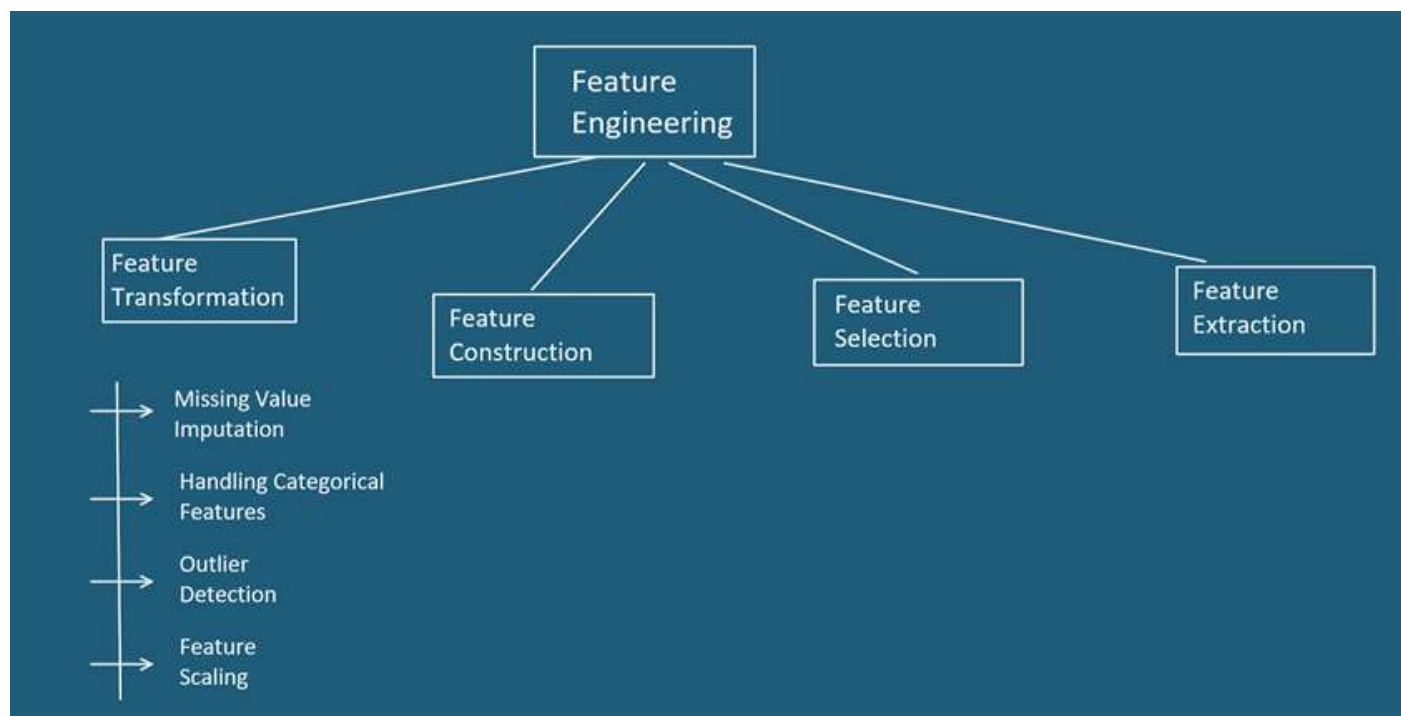
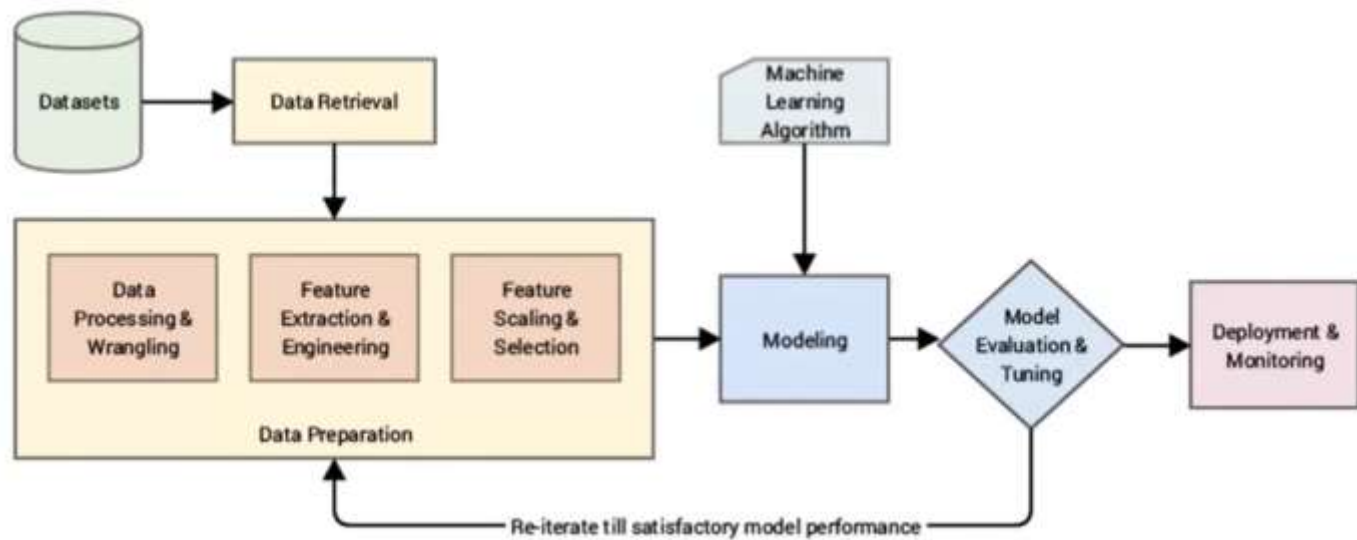
Technical Goal: To find a new, orthogonal coordinate system such that the axes (the Principal Components) are ordered by the amount of data variance they capture.

How it Works:

1. **Find the Direction of Maximum Variance:** The **First Principal Component (PC1)** is a line that passes through the data cloud in the direction where the data is most spread out. This component captures the maximum possible variance (the most information) from the original data.
2. **Find Subsequent Orthogonal Directions:** The **Second Principal Component (PC2)** is a line orthogonal (perpendicular) to PC1, capturing the maximum remaining variance, and so on.
3. **Dimensionality Reduction:** The technique ranks all components by the variance they explain (measured by their corresponding **eigenvalues**). By keeping only the first k components that explain, say, 90% of the total variance, and discarding the rest, you reduce the dimensionality while minimizing information loss.

Key Property	Description
Linear	Components are linear combinations of the original features.
Unsupervised	Does not require or use the target variable (labels) for the transformation.
Variance Maximization	Prioritizes preserving the directions that show the greatest spread in the data.

Feature Engineering:



Feature Engineering is the broad, iterative process of using **domain knowledge** to select, transform, combine, or create variables from raw data to make machine learning models work more effectively. It is a precursor to feature extraction and selection.

- **Goal:** To improve model performance by creating features that better represent the underlying problem.
- **Examples:**
 - **Handling Dates:** Extracting DayOfWeek, Month, or TimeSinceLastEvent from a single timestamp.

- **Combining Features:** Creating a new feature like *Body Mass Index (BMI)* from *Height* and *Weight* ($BMI = Weight / Height^2$)
- **Encoding:** Converting categorical variables (like City or Color) into numerical formats (e.g., One-Hot Encoding).

Feature Selection:

Feature Selection is the process of automatically or manually selecting a subset of the most relevant and non-redundant features (variables, predictors) from a larger dataset to be used in model construction.

The goal is to improve the model's performance, speed, and interpretability by focusing on the most influential input variables and eliminating irrelevant or redundant ones.

Why Feature Selection is Essential:

Selecting the right features is a critical step in the machine learning workflow, offering several benefits:

- **Improved Model Performance:** By removing noisy, irrelevant, or redundant features, the model can focus on the true signal, leading to better generalization and often higher accuracy. It helps mitigate overfitting.
- **Reduced Training Time & Computational Cost:** Fewer features mean less data to process and simpler models, drastically reducing the time required for model training and deployment.
- **Enhanced Interpretability:** A model built on a smaller, well-chosen set of features is much easier for humans to understand, explain, and monitor.
- **Mitigate the Curse of Dimensionality:** It prevents the negative effects of high-dimensional data, where data points become sparse and distances between them lose meaning.

Main Categories of Feature Selection Techniques

- Feature selection techniques are broadly categorized into three main types, based on how they interact with the machine learning model:

Feature Extraction:

Feature Extraction is a crucial technique in machine learning that transforms high-dimensional raw data into a set of new, lower-dimensional features (components) that capture the most relevant information.

Unlike **Feature Selection**, which merely picks a subset of the original variables, feature extraction creates **new features** by combining or transforming the existing ones. This transformation is necessary to reduce data complexity and dimensionality, which helps algorithms run faster and generalize better, especially when dealing with data like images, audio, or text.

Key Goals and Benefits:

1. **Dimensionality Reduction:** It is a powerful antidote to the **Curse of Dimensionality**, reducing the computational cost and memory usage associated with handling a large number of features.
2. **Information Preservation:** It aims to retain the essential information (variance) from the original high-dimensional space in the new, compact feature space.
3. **Noise Reduction:** By focusing on the variance/signal and discarding lower-variance directions, it effectively removes irrelevant noise from the dataset.
4. **Improved Model Performance:** The new, highly informative and uncorrelated features often lead to simpler models that are less prone to overfitting and achieve higher predictive accuracy.

Common Feature Extraction Techniques:

Feature extraction methods can be broadly classified based on the nature of the data and the underlying mathematical principle.

1. Linear Dimensionality Reduction

These methods find a linear combination of the original features to create the new components.

- **Principal Component Analysis (PCA):**
 - **Goal:** Finds orthogonal directions (Principal Components) that maximize the variance of the data.
 - **Type:** Unsupervised (does not use the target variable).
 - **Mechanism:** Identifies the axis that explains the most variance in the data and projects the data onto that axis.
- **Linear Discriminant Analysis (LDA):**
 - **Goal:** Finds directions that maximize the separation *between* different classes while minimizing the variance *within* each class.

- **Type:** Supervised (uses the target variable/class labels).
- **Use:** Often used for classification tasks to improve class separation.

Feature Selection vs. Feature Extraction

Both feature selection and feature extraction are types of dimensionality reduction, but they differ fundamentally in how they achieve it.

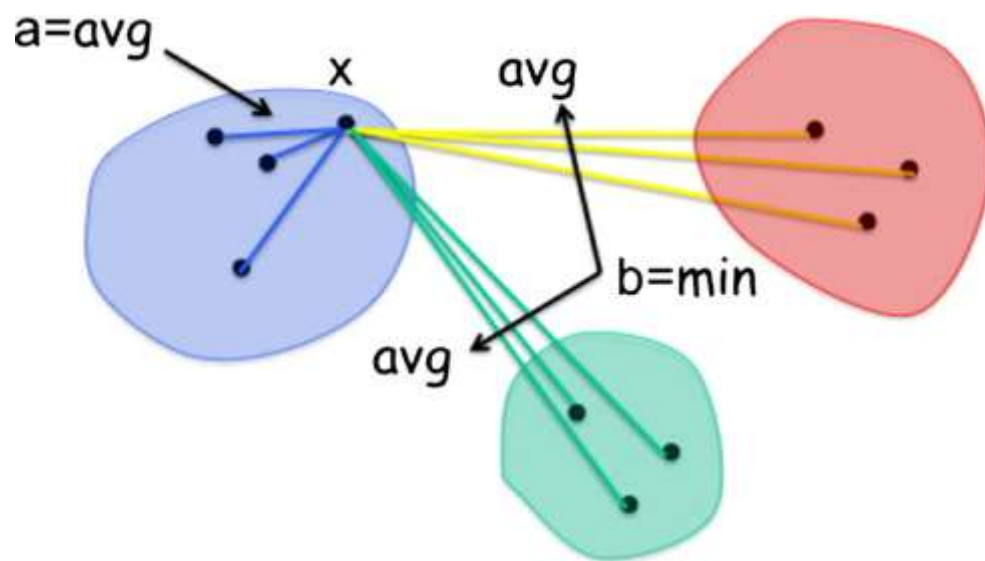
	Feature Selection	Feature Extraction
Method	Chooses a subset of the original features and discards the rest.	Creates a new set of features (components) by transforming or combining the original features.
New Features?	No. The features are a direct subset of the input variables.	Yes. The new components are derived from the old ones.
Interpretability	High. The original features retain their physical meaning.	Low. The new components are abstract linear combinations of the original features.
Examples	Filter methods (Correlation), Wrapper methods (Recursive Feature Elimination), Embedded methods (Lasso/Ridge Regression coefficients).	PCA , Linear Discriminant Analysis (LDA), Singular Value Decomposition (SVD), Autoencoders.

Model Evaluation for Unsupervised Learning: Silhouette Score:

The **Silhouette Score** (or Silhouette Coefficient) is a key metric used for evaluating the quality of clustering results in **unsupervised learning**. It provides a quantitative measure of how well-defined and separated the clusters are by assessing both the **cohesion** within a cluster and the **separation** from neighbouring clusters.

Since unsupervised learning lacks true labels (ground truth), the Silhouette Score acts as an **internal validation metric**, rewarding clusters that are dense and well-separated.

Calculation of the Silhouette Score:



The Silhouette Score is calculated for **each individual data point** and then averaged to get the overall score for the clustering solution. The score for a single data point i , denoted as $s(i)$, is calculated using two main distances:

1. Cohesion ($a(i)$): Mean Intra-Cluster Distance

- $a(i)$ is the **average distance** between data point i and all other points in the **same cluster**.
- It measures how well i is matched to its own cluster. A **small** $a(i)$ value indicates high cohesion (the point is close to its cluster center).

2. Separation ($b(i)$): Mean Nearest-Cluster Distance

- $b(i)$ is the **average distance** between data point i and all points in the **next nearest cluster** (the one i does not belong to, but is closest to).
- It measures how well i is separated from its neighbouring cluster. A **large** $b(i)$ value indicates high separation.

The Silhouette Score $s(i)$ for the individual data point i is then calculated by the formula:

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

The **Overall Silhouette Score** for the entire clustering is the mean of the Silhouette Scores for all data points in the dataset.

Interpretation of the Score

The Silhouette Score ranges from **-1 to +1**.

Score Range	Interpretation	Quality of Clustering
Close to +1	The data point is far from the neighboring clusters and tightly grouped within its own cluster ($b(i) \gg a(i)$).	Strong and Well-Separated
Close to 0	The data point is on or very close to the decision boundary between two neighboring clusters ($b(i) \approx a(i)$).	Overlapping or Ambiguous
Close to -1	The data point is closer to the neighboring cluster than its own assigned cluster ($a(i) > b(i)$).	Misclassified/Incorrect Cluster

Goal: A data scientist aims for a high overall Silhouette Score, typically above 0.5, to indicate a reasonable structure with well-formed clusters.

Example: Using Silhouette Score to Find Optimal k:

One of the most practical uses of the Silhouette Score is determining the optimal number of clusters (k) for algorithms like K-Means.

1. **Iterate:** Run the clustering algorithm multiple times, trying different values for k (e.g., k=2, 3, 4, 5,).
2. **Calculate:** For each value of k, calculate the overall average Silhouette Score.
3. **Select:** The value of k that results in the **highest average Silhouette Score** is typically chosen as the optimal number of clusters, as it represents the clustering solution with the best combination of density and separation.

Example: If k=3 yields a score of 0.72, and k=4 yields a score of 0.61, the clustering solution with k=3 is generally preferred.

Introduction to Scikit-learn (sklearn)

Scikit-learn (sklearn) is the most popular, free, and open-source **Python library** designed specifically for **machine learning** tasks. It's built on top of the essential scientific computing libraries in Python, including **NumPy**, **SciPy**, and **Matplotlib**.

Sklearn provides a wide range of efficient tools for statistical modelling, including classification, regression, clustering, and dimensionality reduction, and it's widely used by data scientists and machine learning engineers for its simplicity, consistency, and depth.

Scikit-learn is not used for data loading, manipulation, or deep learning (like building complex neural networks), but rather for applying models to structured data.

Feature Area	Description	Example Algorithms
Supervised Learning	Training models using labelled data.	Linear Regression, Logistic Regression, Support Vector Machines (SVM), Decision Trees, Random Forests.
Unsupervised Learning	Finding patterns and structure in unlabelled data.	K-Means Clustering, DBSCAN, Hierarchical Clustering.
Model Selection	Comparing, tuning, and validating models.	Cross-Validation, Grid Search for hyperparameter tuning.
Preprocessing	Preparing raw data for modelling.	Scaling (StandardScaler, MinMaxScaler), Encoding (OneHotEncoder).
Dimensionality Reduction	Reducing the number of features.	Principal Component Analysis (PCA).

The Scikit-learn API Philosophy

The strength of sklearn lies in its incredibly **consistent API (Application Programming Interface)**. Nearly all objects—whether they are models, transformers, or selection tools—follow three core interface methods:

1. **estimator.fit():**

- **Purpose:** The core function used to **train** or **learn** the parameters of a model (estimator) from the data.
- **Syntax (Supervised):** estimator.fit(X, y) where X is the feature matrix and y is the target vector.
- **Syntax (Unsupervised/Transformer):** estimator.fit(X)

2. **estimator.predict():**

- **Purpose:** Used after fitting to generate **predictions** on new data.
- **Syntax:** estimator.predict(X_new)
- **Output:** The predicted class labels (for classification) or continuous values (for regression).

3. **estimator.transform() / estimator.fit_transform():**

- **Purpose:** Used by preprocessing and dimensionality reduction objects (like PCA or StandardScaler) to modify the dataset.
- **fit_transform:** Fits the necessary parameters (e.g., mean and standard deviation) to the training data and then applies the transformation, often used for training data.
- **transform:** Applies a previously learned transformation to new data (e.g., test data).

This uniform approach means that once you learn how to use one sklearn model (e.g., Logistic Regression), you essentially know how to use all of them, greatly simplifying the workflow.

Choosing a Scikit-learn Linear Regression Algorithm:

